

Beyond Brittle ACLs: Deterministic Governance and AI Visibility Boundaries in TenantSage

Arthit Pukhampuang

TenantSage Project — Melbourne, Victoria, Australia

Abstract

This paper presents TenantSage, a governance-first infrastructure designed to resolve the security crisis inherent in multi-tenant Retrieval-Augmented Generation (RAG) systems. Traditional RAG architectures rely on static, denormalized Access Control Lists (ACLs) stamped onto vector embeddings — a method that is fundamentally brittle and prone to *permission drift*, where security states in the vector store diverge from the authoritative application source. TenantSage replaces this legacy model with a **Dynamic Authority Resolver (DAR)** architecture.

The DAR architecture facilitates the creation of *AI Blind Spots*, a security paradigm where unauthorized or quarantined data is rendered entirely invisible to AI models at query time through real-time hierarchical resolution. Rather than checking boolean flags, the DAR evaluates cryptographic evidence tokens across a strictly defined relational join-chain. This framework is anchored by two pillars: **Fail-Closed** behavior and **Replay Reconstruction**. Fail-Closed behavior ensures that any structural break in the authority chain — be it a suspended tenant, an expired temporal assignment, or a missing policy link — collapses the query result to zero rows by definition. Replay Reconstruction utilizes an immutable, append-only ledger, cryptographically bound to the system state, to enable the deterministic reproduction of visibility boundaries at any millisecond in history. By decoupling governance from data storage, TenantSage achieves deterministic visibility without the latency and consistency overhead of costly vector re-indexing.

1. Introduction: The Crisis of Permission Drift in RAG Systems

In the current landscape of AI deployment, the intersection of multi-tenancy and vector search has exposed a critical vulnerability: the inability to maintain real-time security synchronization. Standard RAG implementations attempt to secure data by embedding denormalized permissions directly into the vector metadata. This approach creates a *brittle* architecture; any change in user status or document sensitivity requires a background re-indexing job to update the vector store. The resulting latency creates a window of *permission drift*, where AI models may inadvertently access data that has been technically revoked.

The fundamental flaw is architectural, not operational. Embedding spaces do not respect tenant boundaries — semantically similar documents from different tenants cluster together in vector space, and without explicit isolation mechanisms evaluated at query time, nearest-neighbor search can trivially cross those boundaries. The security failure does not occur at the LLM response stage; it occurs earlier, during vector search and document injection into the

prompt context. Once unauthorized content enters the context window, the model fluently synthesizes it into its response — the model is not at fault, but the retrieval boundary has already been breached.^{[1][2]}

This crisis originates from a failure to distinguish between the **Application Domain Model** and the **Governance Infrastructure Model**. Application models are inherently volatile and mutable, optimized for operational performance and destructive updates. Conversely, a Governance Infrastructure Model must be immutable and deterministic. TenantSage is engineered to bridge this gap, ensuring that visibility boundaries are not merely "checked" but are evaluated as cryptographic assertions at the exact moment of retrieval. The goal is to move from discretionary row-level checks to a dedicated execution layer that guarantees the AI model's context window is restricted to authorized evidence items only.

2. Related Work

2.1 Access Control in Vector-Based Retrieval Systems

The OWASP Top 10 for LLM Applications 2025 formally classifies multi-tenant vector database exposure under LLM08:2025 — **Vector and Embedding Weaknesses**, identifying unauthorized access, cross-context leakage, embedding inversion attacks, and data poisoning as the primary threat vectors. The classification specifically notes that vector store queries executed without access control checks — returning documents based solely on embedding distance rather than authorization state — represent a first-class security vulnerability, not a configuration oversight.^{[3][4][5][6]}

Existing approaches to RAG authorization fall into three broad categories. *Pre-ingestion filtering* restricts which documents enter the vector store at index time, but cannot accommodate dynamic permission changes without re-indexing. *Metadata-filter injection* appends tenant or permission predicates to the similarity search query, but relies on denormalized metadata that is subject to drift. *Post-retrieval filtering* evaluates results after retrieval but before prompt assembly, introducing a window during which unauthorized content has technically been accessed by the retrieval layer. TenantSage supersedes all three by relocating authorization to a structural join-chain that is logically antecedent to retrieval itself.^{[7][1]}

2.2 Relationship-Based Access Control

Relationship-Based Access Control (ReBAC), formalized by Bruns et al. (2012) and Cheng et al. (2012), models authorization as a directed graph traversal — a subject's permission to access a resource is determined by the existence of a qualifying path of relationships between them in a system graph. The Attribute-supporting Relationship-Based Access Control model (AReBAC), introduced by Rizvi and Fong (2018), extends this by allowing attribute constraints on vertices and edges along the relationship path, enabling fine-grained authorization policy that combines graph topology with attribute evaluation. TenantSage's DAR join-chain is a relational instantiation of AReBAC: the graph (Tenant → Family → Children → User) is evaluated as a SQL join sequence against live database state, with attribute constraints applied via temporal user assignment windows and family-level policy overrides.^{[8][9][^10]}

2.3 Cryptographic Audit Ledgers

Tamper-evident, append-only audit logging using SHA-256 hash chains — where each entry binds the hash of the previous entry, making any edit, deletion, or reordering detectable — is an established primitive in regulated system design. Scaling strategies for such systems include hash chaining for linear verification, Merkle tree structures for $O(\log n)$ proof generation, digital signature batching over hash aggregates, and external anchoring to RFC 3161-compliant timestamping authorities or public transparency logs for non-repudiable checkpointing. TenantSage's `authority_events` ledger applies this primitive directly within PostgreSQL, with the atomic write path ensuring no observable governance decision exists without a corresponding, hash-chained audit fact.^{[11][12][13][14]}

2.4 Fail-Closed and Zero-Trust Principles in AI Systems

The Zero Trust Architecture principle of deny-by-default — where any action without an explicit ALLOW policy is blocked — has been formally extended to autonomous AI agent deployments by the Cloud Security Alliance (2026), Microsoft (2026), and the U.S. Department of Defense (2026). The Agentic Trust Framework codifies this as *Agentic Zero Trust*: no AI agent action should be trusted by default regardless of prior authentication state. The emerging *suspect-by-default* paradigm (arXiv, 2026) argues further that systems should be designed to reduce their dependency on establishing trust at all, rather than merely moving the trust boundary. TenantSage's Fail-Closed join-chain embodies this principle structurally: authority is not checked and then trusted; it is resolved at the millisecond of query initiation, and the absence of resolution produces zero rows by mathematical definition.^{[15][16][17][18]}

3. The TenantSage Architectural Framework

TenantSage shifts the governance burden from the database schema to a structured hierarchy designed to enforce authority through inheritance and deterministic resolution. The vector store is deliberately reduced to a pure semantic index — a store of meaning, not of permission. All visibility determination is delegated to the DAR engine, which operates on a separate, immutable governance plane.

3.1 Hierarchical Authority Mapping

The system governs all AI operations through a rigorous four-tier hierarchy:

- **Tenant** — The root authority and top-level isolation boundary. No cross-tenant data path exists by construction.
- **Family** — A logical resource container or grouping (e.g., a project or document collection) that serves as the authoritative parent for inherited governance and the primary anchor for policy application.
- **Users** — Principals (human actors or service accounts) requesting evaluated authority.
- **Children** — The leaf resources or individual evidence items (vectors, chunks, records) whose access is governed by the parent hierarchy.

3.2 Resource Taxonomy

The following table maps domain entities to their specific roles within the TenantSage governance infrastructure:

Domain Entity	Infrastructure Concept	Role in Governance
Tenant	Isolation Boundary	Root of ownership; defines hard multi-tenant isolation
Family	Resource Container	Authoritative parent for resource grouping and policy inheritance
Children	Evidence Items	Immutable leaf resources; targets of the visibility boundary evaluation
Users	Principals	Entities requiring cryptographic assertions to access evidence
User Assignments	Scoped Temporal Bindings	Effective-from / effective-to constraints on principal-to-child relationships
Policies	Override Logic	Family-level precedence rules including legal holds and quarantine states
Authority Events	Append-Only Ledger	Cryptographic chain of all resolution events; the audit root

3.3 The DAR Engine

The Dynamic Authority Resolver (DAR) is the core runtime component of TenantSage. The DAR engine walks the relational tree in real time for every retrieval request. Authority is never persisted alongside the data (the vectors); instead, it is resolved through the parent sources at the millisecond a query is initiated. This ensures that the vector store remains a pure representation of data, while the DAR manages the boundaries of its visibility.

The logical separation is absolute: the similarity search index determines *what is semantically relevant*; the DAR determines *what the requesting principal is permitted to see*. These two determinations are structurally independent. The DAR's output — the set of child IDs that pass the authority check — is used to constrain the similarity search result, not to filter it post-hoc. Authorization precedes retrieval, not the reverse.

4. Mechanisms of Deterministic Governance

4.1 Real-Time Inheritance Retrieval

In the TenantSage model, vector data possesses no independent authority. Security is a **transient state** derived from real-time inheritance. When a principal requests data, the DAR executes a five-step resolution process:

1. **Tenant Scope** — Verify the request originates within an active, unrevoked tenant boundary.
2. **Family Context** — Identify the logical container and inherit applicable policy defaults and status.
3. **Authorized Children** — Validate specific user assignments to child resources.
4. **Role/Policy Context** — Evaluate temporal constraints, legal holds, and precedence rules against cryptographic evidence tokens.

5. **Decision & Entry** — Issue the final cryptographic assertion and simultaneously commit the resolution — including a unique `trace_id` — to the `authority_events` ledger as an atomic operation.

The atomicity of Step 5 is non-negotiable. If the ledger write fails, the entire resolution is rolled back. A governance decision that cannot be recorded does not occur. This structural invariant ensures the ledger is complete by construction, not by discipline.

4.2 Fail-Closed Behavior

TenantSage enforces a **Strict Fail-Closed** architecture. This is not merely a policy — it is a structural property of the relational join-chain:

```
Tenant (active) → Family (not quarantined) → Children (visible) → User (assigned, wi-
```

Because authority is resolved through an active chain of joins, if any single link is missing, revoked, or suspended, the SQL result set is empty by definition. No code path exists that returns data on an incomplete chain. This prevents Fail-Open scenarios caused by ambiguous metadata or missing flags: if the chain of evidence is incomplete, the system collapses to secure denial without any application-level error handling being required.

The asymmetry of this design is deliberate. Adding a new authorization is an explicit, audited act. Removing authorization — or having it expire — is passive and automatic; it requires no administrative action. The system is biased toward denial at rest and toward access only under active, complete, cryptographic assertion.

4.3 Temporal and Policy Constraints

Authority is further refined through `user_assignments` and policies. User assignments utilize `effective_from` and `effective_to` timestamps to ensure visibility is restricted to specific windows. Policies provide family-level overrides — including legal holds and quarantine statuses — which suppress entire resource containers regardless of individual user permissions.

This temporal constraint model directly addresses the *index-time staleness* problem: access captured at ingestion that continues to answer queries long after revocation, including for off-boarded personnel whose credentials remain embedded in vector metadata. In TenantSage, the expiry of a `user_assignment` row causes the join-chain to break at that link. No re-indexing is required. No background job is scheduled. The permission change takes effect at the next DAR resolution, which is the next query — a latency measured in milliseconds, not minutes.

5. Engineering AI Blind Spots

5.1 Direct Lineage Suppression

TenantSage allows administrators to create instantaneous **AI Blind Spots**. By flagging a Family-level container for quarantine, the DAR engine dynamically suppresses all associated child resources at the join-chain level. Because this suppression occurs at the resolution layer, no metadata within the vector database is modified, and no re-indexing is required. The data simply vanishes from the AI's retrieval path — a semantically complete suppression with zero operational overhead.

The quarantine state is applied to a single row in the `families` table. Its effect propagates immediately to all children of that family, to all users assigned to those children, and to all future DAR resolutions involving any of those resources. The suppression requires no enumeration of affected records, no batch update, and no consistency window.

5.2 Eligible Evidence Boundaries

The DAR establishes a hard **Eligible Evidence Boundary**. The AI model's retrieval window is strictly limited to evidence items that successfully pass the runtime authority check and possess the required cryptographic tokens. If the DAR does not produce a record for a child resource, the AI model has no technical path to process that information. The context window is guaranteed to contain only authorized content.

This property — that the AI model's knowledge during a query execution is bounded by the DAR's output — is the central security claim of TenantSage. It cannot be circumvented by crafting a semantically similar query, by manipulating prompt context, or by any mechanism that operates at the LLM layer. The boundary is enforced at the infrastructure layer, below the model.

5.3 Efficiency Versus Re-indexing

By evaluating relational logic at the moment of the query, TenantSage eliminates the performance bottlenecks associated with eventual consistency in vector stores. Global governance updates — tenant suspensions, quarantines, legal holds, user off-boarding — take effect immediately across all AI query points without background synchronization latency. The governance state is always current because it is always evaluated live, never cached.

6. Integrity and Replay Reconstruction

6.1 The Immutable Audit Ledger

All governance activity in TenantSage is captured by the `authority_events` table, which functions as an append-only ledger. Unlike application models that use destructive updates (mutating a status field and losing historical state), TenantSage records every resolution, suppression, and hold event as a discrete, immutable entry. The `trace_id` field serves as the connective tissue, linking every specific AI query execution back to its corresponding governance decision.

No entry in `authority_events` is ever updated or deleted. Administrative corrections are themselves new entries — a correction event that references the original `trace_id`. This means the ledger is not merely a log of what happened; it is the authoritative, complete record of what the governance engine decided, and when, and on what evidence.

6.2 State-Checksumming and Cryptographic Binding

To satisfy the most rigorous transparency requirements, TenantSage implements a **Tamper-Evident Chronology**. Each entry in the authority ledger is cryptographically bound to the previous entry using a `system_sequence` and a `state_checksum`. This creates a functional chain of execution — analogous to block headers — where the integrity of the audit log is inextricably linked to the state of the governance engine at the time of the event.

The full write path for an authority event is:

1. Receive resolution request with principal identity and resource target
2. Validate join-chain integrity across Tenant → Family → Children → User
3. Evaluate temporal constraints and policy overrides
4. Retrieve the hash of the most recent entry in the `authority_events` chain
5. Canonicalize the new event record concatenated with the previous hash
6. Compute SHA-256 hash of the canonicalized payload
7. Write the event, hash, `system_sequence`, and `trace_id` to `authority_events` in a single atomic transaction
8. Return the `trace_id` and cryptographic assertion to the calling retrieval layer

If the hash of any historical entry is recomputed and does not match the `state_checksum` stored in its successor, the chain is broken and tampering is detected at the exact event where it occurred. Because this check can be executed on demand with a single traversal of the ledger, any compliance auditor, regulator, or security team can independently verify the integrity of the entire governance history without requiring access to application code or runtime state.
[^13]

For deployments requiring non-repudiable external verification, the chain head can be periodically anchored to an RFC 3161-compliant timestamping authority or a public transparency log. This provides an external checkpoint that the operator cannot retroactively alter, satisfying the highest tier of audit requirements.[^12]

6.3 Point-in-Time Reconstruction

The combination of an append-only ledger and cryptographic sequencing enables **Replay Reconstruction**. Administrators can reproduce the exact state of authority permissions at any precise millisecond in history. By querying `authority_events` with a bounded timestamp range and walking the hash chain to verify integrity, the system can prove deterministically why a specific retrieval was permitted or denied at a specific time.

This is not an approximation or a reconstruction from logs — it is a replay of the exact governance state. The ledger records not merely the decision (permit/deny), but the full evidence chain on which the decision was based: the tenant state, the family status, the user assignment record, the policy evaluation result, and the cryptographic token issued. A regulator asking *"why did this AI model access this document at 14:37:22.841 on this date?"* receives a verifiable, mathematically consistent answer derived from the chain itself.

7. Compliance Alignment

TenantSage's architecture is designed to satisfy the governance requirements of the major regulatory frameworks applicable to AI systems operating in multi-tenant enterprise environments.

The EU AI Act (Regulation 2024/1689) mandates, under Article 12, that high-risk AI systems technically allow for the automatic recording of events over the lifetime of the system, including the reference database against which input data was checked and the input data for which a retrieval match was made. Article 13 further requires that high-risk AI systems be designed so that deployers can interpret system outputs and use the system appropriately, with logging mechanisms sufficient to ensure proper functioning and compliance. Article 12 additionally specifies a minimum 6-month automatic event logging retention period with full auditability. The TenantSage `authority_events` ledger with `trace_id`-linked query-to-decision traceability satisfies all three requirements natively.^{[19][20][^21]}

The SOC 2 Trust Services Criteria (specifically CC7.2, CC7.3 for monitoring and CC6.1 for logical access) require that logs exist, are protected from unauthorized modification, are retained for an appropriate period, and are monitored for anomalies. TenantSage's hash-chained, append-only ledger addresses the tamper-evidence requirement structurally; administrative access controls on the `authority_events` table address preventability; and the `system_sequence` monotonic ordering addresses the log integrity requirement.^[^13]

OWASP LLM08:2025 identifies missing tenant isolation in multi-tenant RAG systems as a first-class vulnerability, specifically noting that vector store configurations that do not enforce tenant boundaries allow one tenant's queries to retrieve another tenant's documents through semantic similarity matching. TenantSage's Fail-Closed join-chain, rooted at the Tenant isolation boundary, makes cross-tenant retrieval structurally impossible — not merely inadvisable.^[^4]

Framework	Specific Requirement	TenantSage Mechanism
EU AI Act Art. 12 ^[^21]	Automatic event logging; reference database recording; minimum 6-month retention	Immutable <code>authority_events</code> ledger; <code>trace_id</code> per query; append-only non-destructive retention
EU AI Act Art. 13 ^[^19]	Transparency of operation; logging mechanisms for compliance	Hash-chained audit trail; Point-in-Time Reconstruction; Replay capability
SOC 2 CC6.1 / CC7.2 / CC7.3 ^[^13]	Tamper-resistant logs; anomaly monitoring; logical access protection	SHA-256 hash chain; chain-break detection; PostgreSQL-level write constraints

Framework	Specific Requirement	TenantSage Mechanism
OWASP LLM08:2025 ^[^4]	Tenant isolation; permission-aware retrieval; cross-context prevention	Fail-Closed join-chain; Eligible Evidence Boundary; AI Blind Spots
Zero Trust / Deny-by-Default ^[^16]	No implicit trust; explicit ALLOW required; assume breach	Relational join-chain collapses to zero rows on any incomplete link
GDPR (Art. 5, 22)	Demonstrable access control; right-to-explanation; data minimization	Replay Reconstruction; deterministic per-query audit record

8. Conclusion

The transition from discretionary row-level checks to evaluated cryptographic assertions marks a paradigm shift in AI security. The TenantSage DAR architecture resolves the security-latency trade-off that has historically forced RAG system designers to choose between real-time permission accuracy and operational performance. By implementing real-time inheritance across a relational join-chain and pairing every governance decision with an atomic, hash-chained ledger entry, TenantSage delivers both properties simultaneously.

Three foundational properties collectively produce a governance posture that exceeds what is achievable through any combination of metadata filtering, vector re-indexing, or post-retrieval policy enforcement. **Fail-Closed** structural denial eliminates false-positive authorization by construction. **AI Blind Spots** via resolution-layer suppression achieve instantaneous, zero-overhead visibility removal without touching the vector store. **Replay Reconstruction** via cryptographic ledger chaining provides a verifiable, deterministic audit trail that satisfies the most stringent regulatory requirements in force.

TenantSage does not make the vector store more secure. It makes the vector store *irrelevant* to the security determination. Authority lives exclusively in the relational join-chain. The AI model's retrieval window is bounded entirely by what the DAR asserts — and the DAR's assertions are permanently, verifiably, and immutably recorded.

Appendix: Governance Schema Primitives

The following entities constitute the core of the TenantSage DAR pipeline and define its computational execution:

- **Tenants** — Functions as the root authority and primary isolation constraint in the DAR join-chain. Defines hard multi-tenant boundaries that cannot be bridged by any downstream permission. A suspended tenant collapses the entire join-chain for all principals and resources within that boundary.
- **Families** — Acts as the authoritative parent for resource grouping and the primary anchor for policy inheritance. Quarantine state applied at this level propagates to all child resources via the join-chain without requiring any enumeration of affected records.
- **Children** — Represents the immutable leaf resources and evidence items subjected to visibility boundary evaluation. Children carry no independent authority; authority flows exclusively through the parent hierarchy.

- **Users** — Identifies the principal or service account requesting authority assertions. Principals must have active, non-expired assignments to specific child resources within an active family and tenant to receive a positive DAR resolution.
- **User Assignments** — Provides the scoped mapping and temporal (`effective_from` / `effective_to`) validity constraints for specific user-to-child relationships. Expired assignments cause immediate collapse of the join-chain at that link without requiring any data modification.
- **Policies** — Provides the override logic and precedence rules (e.g., legal holds, quarantine flags) for join-chain evaluation. Policy-level suppression overrides all individual user assignments for resources within the governed Family and takes effect at the next DAR resolution.
- **Authority Events** — Functions as the append-only ledger and cryptographic chain for historical replay and query-to-decision traceability. Each entry is SHA-256 hash-chained to its predecessor and atomically committed in the same database transaction as the governance decision it records. No governance decision exists in an observable runtime state without a corresponding, hash-chained, immutable audit fact.

References

1. [Nhi Mgmt Group Analysis](#) - Building a production-style RAG pipeline with multi-tenant permissions depends on matching retrieval...
2. [Governance and Access Control for Multi-Tenant RAG: Preventing Data Leakage Across Tenants](#)
3. [The OWASP Top 10 for LLM applications: What developers ...](#) - How LLMs leak data, get hijacked, and turn friendly inputs into exploits, and why most of the defenses...
4. [LLM08:2025 Vector and Embedding Weaknesses - OWASP Top 10 ...](#) - Vector and Embedding Weaknesses is the eighth risk in the OWASP Top 10 for LLM Applications 2025. It...
5. [LLM08:2025 Vector and Embedding Weaknesses](#) - Vectors and embeddings vulnerabilities present significant security risks in systems utilizing Retri...
6. [Mitigating Vector and Embedding Weaknesses - Ackuity](#) - Explore the OWASP Top 10 risks for LLM applications in 2025, highlighting critical security threats ...
7. [Multi-Tenant RAG Data Isolation: The 2026 Enterprise Architecture ...](#) - Learn how to architect secure RAG pipelines by syncing source-system permissions into vector databas...
8. [Comparison of Access Control Approaches for Graph- ...](#)
9. [22 Learning Relationship-Based Access Control Policies from ...](#)
10. [Enhancing Relationship-Based Access Control Policies with ...](#)
11. [pametan/audit-log: Tamper-evident, append-only ...](#) - Tamper-evident, append-only audit logging for Node: SHA-256 hash chain (optional HMAC), RBAC-aware e...
12. [Audit trails and tamper evidence — Scaling Strategies](#) - Audit trails and tamper evidence — Scaling Strategies — Practical Guide (Feb 22, 2026) Audit trails ...
13. [OpenClaw Immutable Audit Log: Build a Tamper-Evident ...](#) - Build a tamper-evident audit trail for OpenClaw using SHA-256 hash chains, structured JSONL, and off...

14. [Audit & Compliance - Agent Governance Toolkit](#) - Governance, trust, identity, and compliance for AI agents
15. [\[PDF\] Careful adoption of agentic AI services - Department of War](#) - • Embed agentic AI systems with fail-safe defaults and containment mechanisms that limit the blast r...
16. [Assume Breach: Agent...](#) - Apply zero trust principles to autonomous AI agents. Learn how to verify every tool call, enforce le...
17. [The End of Trust: How Agentic AI Breaks Security Assumptions](#) - arXiv - We introduce the Infinite Impostor, an attack model in which an autonomous agent interposes itself b...
18. [The Agentic Trust Framework: Zero Trust Governance for AI Agents](#) - Traditional Zero Trust: No user or system should be trusted by default, regardless of location or ne...
19. [AI Act Service Desk - Article 13: Transparency and provision of ...](#)
20. [AI Transparency and Auditability Requirements Under EU AI Act](#) - EU AI Act transparency requirements under Art. 13 and Art. 52: what AI providers and deployers must ...
21. [Article 12: Record-Keeping](#)